Line Following and Obstacle Clearing

Line Following and Obstacle Clearing

1. Course Content
2. Preparation
3. Case Demonstration
4. Color Calibration
5. Core Source Code Analysis

1. Course Content

This feature implements the program acquiring images through the camera and recognizing the color of the line. The program controls the car to move along the line. During this period, the LiDAR scans for obstacles on the path, and if an obstacle is encountered, the car stops; if an AprilTag appears on the route, it adjusts its pose, lowers its gripper to grasp the AprilTag, places it aside, and then continues line following.

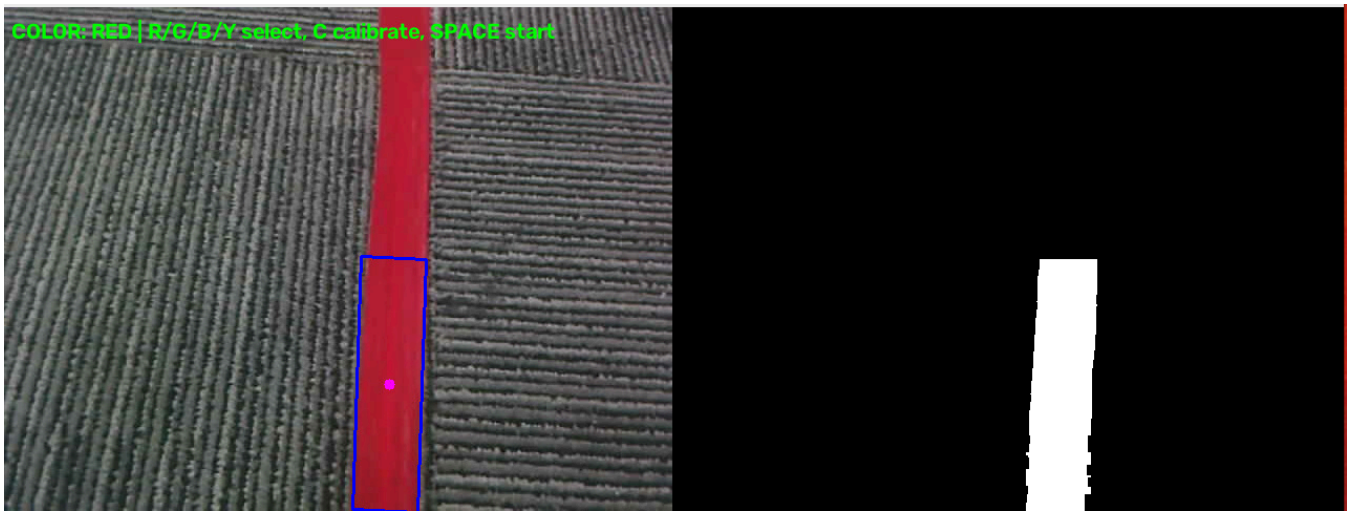
2. Preparation

The wooden block used in this section is: **30x30x30mm AprilTag wooden block**

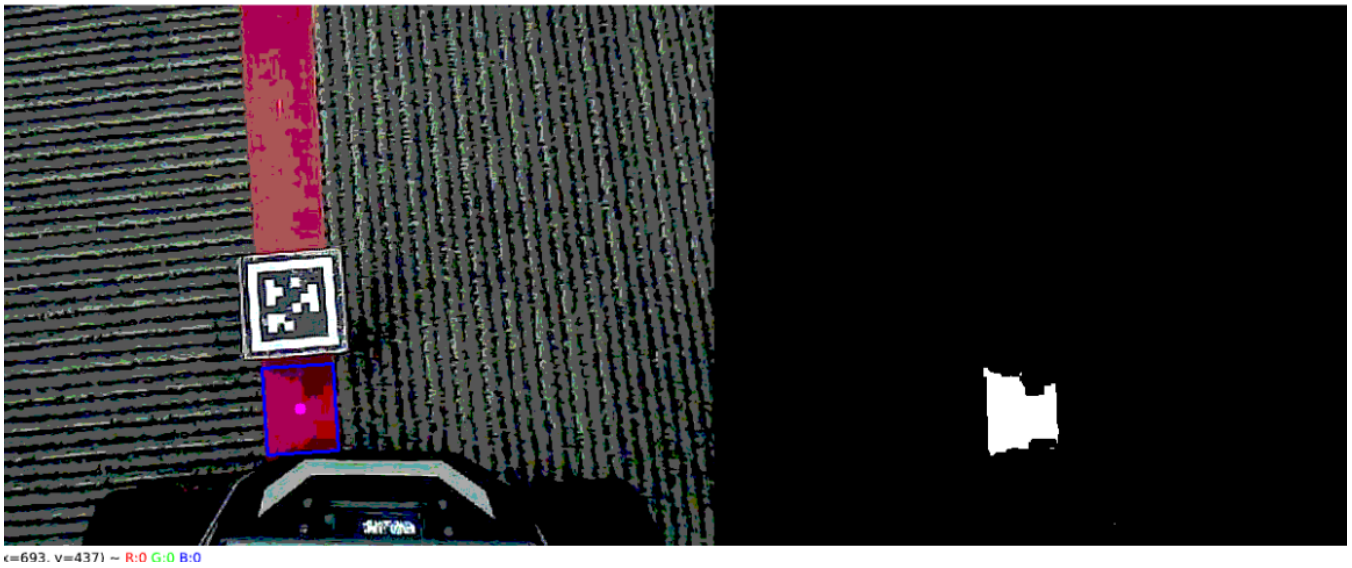
```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=follow_line_clear
```

3. Case Demonstration

- Press R, G, B, or Y to select the color to follow, and press the spacebar to start following the path

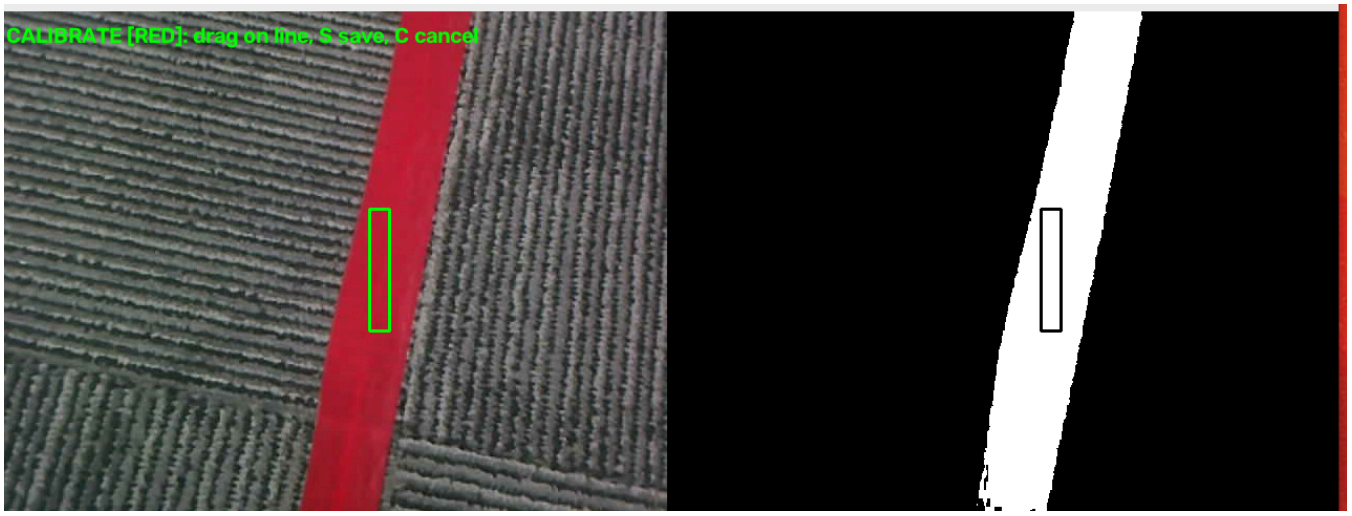


If an AprilTag block appears on the ground ahead, the car pauses following, first removes the AprilTag block, and then continues line following

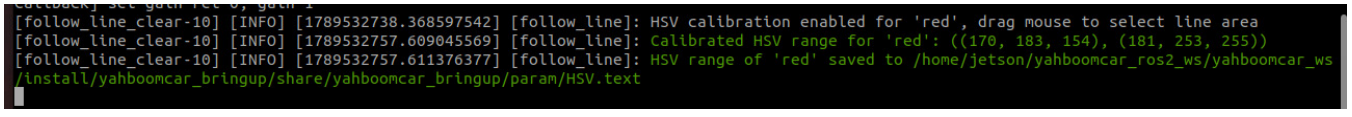


4. Color Calibration

- If the current color recognition is not good, press C to enter color calibration mode, use the mouse to draw a box on the color, and then check the color recognition effect in the binarized image



- After calibration is complete, press the S key to save the calibration result to the parameter file



5. Core Source Code Analysis

- Program path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/arm_demo/follow_line_clear.py
```

_iterative_align function (iterative alignment):

This function adjusts the chassis position through multiple iterations to make the robot aim at the target AprilTag, ensuring that the relative position between the robotic arm end-effector and the target meets the preset conditions during grasping.

- `tolerance = 0.017`: alignment tolerance (meters); when the position error is less than this value, alignment is considered complete
- `required_stable = 2`: alignment must be detected as stable for 2 consecutive times before returning, preventing a single accidental satisfaction of the condition
- `max_iterations = 12`: maximum number of iterations; after exceeding this, alignment is abandoned and the current pose is returned
- `max_step = 0.04`: maximum distance of a single move (meters), limiting the adjustment amplitude each time to avoid overshoot
- `_get_fresh_tag_data(tag_id)`: obtains the latest AprilTag detection result and pose data
- `delta_x = pose[0] - grasp_distance_x`: calculates the deviation between the current x direction and the expected grasping distance
- `y_needs = abs(pose[1]) > grasp_distance_y + tolerance`: determines whether the y-direction deviation exceeds the tolerance
- `stable_count`: stability counter; increments when the alignment condition is satisfied consecutively, otherwise resets
- `move_by_odom(distance=..., direction=...)`: calls the chassis odometry Action to move the specified distance; the direction is determined by the sign of the deviation (forward/left/right)
- After each move, wait 2 seconds for the chassis to stabilize, then re-detect the pose

_execute_grasp function (grasping thread wrapper):

- Calls `_do_grasp` to execute the actual grasping logic, catches exceptions, and logs them
- `finally: self._reset_grasp_state()`: regardless of success or failure, resets the grasping state at the end

_do_grasp function (execute grasping):

This function is the main control function of the grasping flow, sequentially completing alignment, angle calculation, inverse kinematics solution, robotic arm motion, gripper closing, placement, and other steps.

- `self.arm_kinematics`: checks whether the kinematics solver is available; if not, skips grasping
- `_iterative_align(tag_id)`: calls the iterative alignment function to obtain the aligned AprilTag pose
- `_get_fresh_tag_data(tag_id)`: re-obtains the latest detection result after alignment is complete, and extracts the AprilTag's yaw angle (yaw)
- `normalize_angle_to_0_45(rvec_yaw_deg)`: normalizes the yaw angle to the range 0°~45°, used to determine the gripper rotation direction
- `grasp_offset`: grasping offset `[gx, gy, gz, pitch]`, compensating for the difference between the camera-detected position and the actual grasping position
- `arm_kinematics.inverse_kinematics(...)`: calls the inverse kinematics solver, taking the target position (camera pose + grasping offset) and pitch angle as input, and outputting 5 joint angles
- `result.approximate` / `result.adjusted_pitch`: handles the cases of approximate solutions and

automatic pitch angle adjustment, outputting warning logs

- `joints[0:3] = round(ik_joints[0:3])`: rounds the J1~J4 joint angles obtained from the inverse solution to integers (the low-level firmware only accepts integers)
- `joints[4] = int(90 + angle_normalized)`: the J5 joint angle is calculated from the normalized yaw angle, so that the gripper rotation aligns with the AprilTag direction
- `joints[5] = 35`: the J6 gripper angle is set to 35° (open state)
- `pubSix_Arm(joints)`: publishes the 6-joint control command, and the robotic arm moves to the grasping pose
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=gripping_angle, time=1500))`: controls the gripper to close individually
- `DROP_JOINTS.get(latest_tag.tag_id, DEFAULT_DROP_JOINTS)`: obtains the placement joint angles from the preset dictionary according to the AprilTag ID; if there is no corresponding ID, uses the default value
- `down_joint.append(self.gripping_angle)`: appends the gripper angle to the placement joint list
- `pubSix_Arm(down_joint)`: the robotic arm moves to the placement position
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))`: opens the gripper to release the object
- `pubSix_Arm(observation_pose)`: returns the robotic arm to the observation pose, ready to continue line following

```
    def _iterative_align(self, tag_id):
        tolerance = 0.017
        required_stable = 2
        max_iterations = 12
        max_step = 0.04
        stable_count = 0
        for iteration in range(max_iterations):
            if not rclpy.ok():
                return None
            time.sleep(0.5)
            tag, pose, _ = self._get_fresh_tag_data(tag_id)
            if tag is None or pose is None:
                self.get_logger().warn(f"[Iter {iteration+1}] tag {tag_id} not visible")
                stable_count = 0
                continue
            delta_x = pose[0] - self.grasp_distance_x
            x_needs = delta_x > tolerance
            y_needs = abs(pose[1]) > self.grasp_distance_y + tolerance
            if not x_needs and not y_needs:
                stable_count += 1
                if stable_count >= required_stable:
                    self.get_logger().info("Chassis alignment stable, returning latest pose
for IK")
                    return pose
                time.sleep(0.3)
                continue
            stable_count = 0
```

```
            if x_needs:
                move_x = min(delta_x, max_step)
                self.move_by_odom(distance=move_x, direction='forward')
            if y_needs:
                move_y = min(abs(pose[1]) - self.grasp_distance_y, max_step)
                direction = 'left' if pose[1] > 0 else 'right'
                self.move_by_odom(distance=move_y, direction=direction)
            time.sleep(2.0)
        self.get_logger().warn(f"Alignment did not converge after {max_iterations}
iterations")
        _, pose, _ = self._get_fresh_tag_data(tag_id)
        return pose

    def _execute_grasp(self, tag_id):
        self.get_logger().info(f"Grasp thread started for tag ID {tag_id}")
        try:
            self._do_grasp(tag_id)
        except Exception as e:
            self.get_logger().error(f"Grasp thread exception: {e}")
        finally:
            self._reset_grasp_state()

    def _do_grasp(self, tag_id):
        if self.arm_kinematics is None:
            self.get_logger().error("ArmKinematics not available, skip sorting")
            return
        final_pose = self._iterative_align(tag_id)
        if final_pose is None:
            self.get_logger().error("Iterative alignment failed")
            return
        time.sleep(0.5)
        latest_tag, _, rvec_yaw_deg = self._get_fresh_tag_data(tag_id)
        if latest_tag is None:
            self.get_logger().error("Tag lost after alignment")
            return

        angle_normalized = normalize_angle_to_0_45(rvec_yaw_deg)
        self.get_logger().info(f"Tag rvec yaw: {rvec_yaw_deg:.1f}° -> normalized:
{angle_normalized:.1f}°")
        mx, my, mz = final_pose
        gx, gy, gz = self.grasp_offset[0], self.grasp_offset[1], self.grasp_offset[2]
        self.get_logger().info(f"camera measure pose : x={mx:.3f}, y={my:.3f}, z={mz:.3f};IK
target with grasp_offset: x={mx+gx:.3f}, y={my+gy:.3f}, z={mz+gz:.3f}, pitch=
{self.grasp_offset[3]:.1f}°")

        result: KinResult = self.arm_kinematics.inverse_kinematics(
            target_x=mx + gx,
            target_y=my + gy,
            target_z=mz + gz,
            target_pitch=self.grasp_offset[3],
            seed_angles_deg=None,
        )
        if not result.success:
```

```
            self.get_logger().error(f"Grasp IK failed: {result.message}")
            return
        if result.approximate:
            self.get_logger().warn(Fore.YELLOW+f"IK using approximate solution, position
error = {result.position_error:.4f} m"+Fore.RESET)
        if result.adjusted_pitch is not None:
            self.get_logger().warn(Fore.YELLOW+f"IK using pitch sweep, adjusted pitch =
{result.adjusted_pitch:.1f}°"+Fore.RESET)

        ik_joints = result.data
        joints = [0, 0, 0, 0, 0, 0]
        joints[0] = round(ik_joints[0])
        joints[1] = round(ik_joints[1])
        joints[2] = round(ik_joints[2])
        joints[3] = round(ik_joints[3])
        joints[4] = int(90+angle_normalized)
        joints[5] = 35

        self.get_logger().info(f"Grasp joints: {joints}")
        self.pubSix_Arm(joints)
        time.sleep(3.5)
        self.SingleJoint_pub.publish(ArmJoint(id=6, joint=self.gripping_angle, time=1500))
        time.sleep(2.0)

        down_joint = list(DROP_JOINTS.get(latest_tag.tag_id, DEFAULT_DROP_JOINTS))
        if latest_tag.tag_id not in DROP_JOINTS:
            self.get_logger().warn(f"No dedicated drop pose for tag {latest_tag.tag_id},
using default")
        down_joint.append(self.gripping_angle)
        self.pubSix_Arm(down_joint)
        time.sleep(2.5)
        self.SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))
        time.sleep(2.0)
        self.pubSix_Arm(self.observation_pose)
        time.sleep(2.5)
```

- Parameter file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

Parameter Description:

- `apriltag_grasp_distance: [0.1, 0.0]`: the expected grasping distance during chassis alignment `[x, y]` (meters); x is the front-back direction distance, and y is the left-right direction offset
- `tag_size: 0.02`: the physical side length of the AprilTag (meters), used for pose estimation
- `grasp_offset: [0.00, 0.0, 0.015, 85.0]`: grasping offset `[x, y, z, pitch]`, compensating for the error reaching the target point caused by servo backlash and link rigidity degradation; pitch=85° means the gripper grasps almost vertically downward
- `gripping_angle: 140`: the angle (degrees) when the gripper is closed; the larger the value, the tighter

the gripper

- `observation_pose: [90, 115, 3, 0, 90, 30]`: the 6 joint angles (degrees) of the observation pose; the robotic arm maintains this pose during line following
- `tag_confirm_frames: 1`: the number of frames a tag must be stably detected consecutively before pausing line following, preventing false triggering
- `tag_trigger_distance: 0.3`: the maximum distance (meters) to trigger sorting; when the AprilTag distance exceeds this value, continue line following to approach, and when it is less than this value, pause line following and start grasping
- `grasp_cooldown: 0.5`: the cooldown time (seconds) before re-detecting tags after a sorting round is complete, preventing repeated triggering

```
follow_line:
  ros__parameters:
    apriltag_grasp_distance: [0.1, 0.0]     # x,y axis alignment distance (meters)
    tag_size: 0.02                          # AprilTag side length (meters)
    grasp_offset: [0.00,0.0,0.015,85.0]     # x,y,z axis offset distance and included angle,
compensating for the error reaching the target point caused by servo backlash and link
rigidity degradation
    gripping_angle: 140                     # gripper grasping angle, in degrees, integer
    observation_pose: [90,115,3,0,90,30]
    tag_confirm_frames: 1                   # number of frames a tag must be stably detected
consecutively before pausing line following
    tag_trigger_distance: 0.3              # maximum distance to trigger sorting (meters);
if exceeded, continue line following to approach
    grasp_cooldown: 0.5                     # cooldown time (seconds) before re-detecting
tags after a sorting round is complete
```